

# FitBuddy – AI Fitness Plan Generator using Gemini Models

## PHASE-WISE PROJECT DOCUMENTATION

*B.Sc. Computer Science Project*

## **PHASE 1 – BRAINSTORMING & IDEATION**

### **1.1 Introduction**

FitBuddy is a web-based AI fitness planning application designed to generate personalized workout plans and nutrition or recovery tips based on a user's fitness goals. The application uses FastAPI for the backend, Jinja2 and HTML for the interface, SQLite with SQLAlchemy for data storage, and Google Gemini models for AI-powered plan generation.

### **1.2 Problem Identification**

Creating a suitable fitness routine can be difficult when users have different goals, ages, weights, and preferred workout intensities. A general workout plan may not suit every user. Users also need simple nutrition and recovery guidance and a way to provide feedback when a generated plan needs adjustment.

### **1.3 Proposed Solution**

FitBuddy provides a single web application where users can enter their fitness information and receive a personalized 7-day workout plan. The system can also provide nutrition or recovery tips and update the workout plan based on user feedback.

### **1.4 Objectives**

The main objectives are to generate personalized workout plans, provide nutrition and recovery guidance, support feedback-based plan updates, store user and plan information, and provide an administrative or coach dashboard for viewing stored users and plans.

### **1.5 Target Users**

The system is intended for users who want personalized fitness guidance and for administrators or coaches who need to view stored users and generated plans.

### **1.6 Key Features**

User fitness input, AI-generated 7-day workout plan, nutrition/recovery tip, feedback-based plan update, SQLite data storage, FastAPI routes, Jinja2 web pages, and a view-all-users dashboard.

### **1.7 Expected Outcome**

The expected outcome is a functional web application that combines a simple user interface, backend API services, database storage, and Gemini-based AI generation to provide personalized fitness planning.

## **PHASE 2 – RESEARCH & REQUIREMENT ANALYSIS**

### **2.1 Introduction**

This phase identifies the functional requirements, technical prerequisites, user scenarios, and technologies needed to develop FitBuddy.

### **2.2 Functional Requirements**

The application should accept user details such as age, weight, fitness goal, and workout intensity; generate a 7-day plan; provide nutrition or recovery guidance; accept feedback; update a workout plan; save information to the database; and allow viewing of stored users and plans.

### **2.3 User Scenarios**

A user enters fitness details and requests a plan. The system generates the plan using Gemini. The user can provide feedback, after which the system can update the workout plan. The user can also request a nutrition or recovery tip. An administrator or coach can view users and plans through the dashboard.

### **2.4 Technical Requirements**

The project specification identifies Python, FastAPI, Uvicorn, Jinja2, SQLAlchemy, SQLite, python-multipart, and Google Generative AI/Gemini integration as the main technologies and dependencies.

### **2.5 Development Environment**

The project can be developed in Visual Studio Code using a Python virtual environment. The specification provides installation through pip and local execution using Uvicorn.

### **2.6 Expected Outcome**

At the end of this phase, the development requirements, user flow, technology stack, and major application functions are clearly identified.

## PHASE 3 – SYSTEM DESIGN & ARCHITECTURE

### 3.1 Introduction

FitBuddy follows a web application architecture in which the frontend communicates with a FastAPI backend. The backend handles application routes, database operations, and Gemini AI integration.

### 3.2 System Architecture

The main components are HTML/Jinja2 frontend, FastAPI backend, Gemini AI services, and SQLite/SQLAlchemy database storage. The frontend sends user requests to FastAPI, the backend processes the request and communicates with Gemini when AI generation is required, and the generated information is stored or displayed.

### 3.3 Project Structure

The specification identifies folders and files for the FastAPI application, templates, database/model logic, routes, AI integration, and environment configuration. The main templates include index, result, and all\_users pages.

### 3.4 AI Integration Points

Gemini integration is used for workout generation, workout-plan updates based on feedback, and nutrition or recovery tips. The specification identifies separate functions for these AI operations.

### 3.5 Database Design

SQLite is used for local data storage and SQLAlchemy is used for database interaction. User details and generated plans are stored so that information can be retrieved through application functions and the dashboard.

### 3.6 Expected Outcome

A clear architecture is established so that frontend, backend, AI, and database components can work together as one application.

## PHASE 4 – AI/GEMINI INTEGRATION & BACKEND DEVELOPMENT

### 4.1 Introduction

This phase focuses on implementing the core backend and connecting the application with Gemini models for AI-powered fitness planning.

### 4.2 Workout Generation

The `generate_workout_gemini` function is responsible for generating a personalized workout plan from the user's fitness information. The intended output is a 7-day workout plan.

### 4.3 Nutrition & Recovery Tip

The `generate_nutrition_tip_with_flash` function is used to generate a nutrition or recovery-related tip using the Gemini integration described in the project specification.

### 4.4 Feedback-Based Update

The `update_workout_plan` function supports the feedback loop. User feedback can be provided so that the generated workout plan can be updated according to the feedback.

### 4.5 FastAPI Routes

The project specification identifies routes including the home page, `/generate-workout`, `/submit-feedback`, and `/view-all-users`. These routes connect the user interface with backend processing and database operations.

### 4.6 Backend Data Handling

Pydantic schemas are used for request data, while database functions are used to save, retrieve, and update user and workout information.

### 4.7 Expected Outcome

A working backend is established with FastAPI routes, Gemini-based AI functions, request handling, and database interaction.

## **PHASE 5 – FRONTEND DEVELOPMENT**

### **5.1 Introduction**

This phase develops the user-facing interface through HTML and Jinja2 templates. The interface is designed to allow users to enter fitness information and view generated results.

### **5.2 Home Page**

The index page provides the main input interface for collecting the required fitness information such as age, weight, goal, and workout intensity.

### **5.3 Result Page**

The result page displays the generated workout plan and related information returned by the backend.

### **5.4 Feedback Interface**

The frontend supports submission of user feedback so that the backend can process the feedback and update the workout plan.

### **5.5 All Users Page**

The all\_users.html page is used for the administrative or coach dashboard to view stored users and plans.

### **5.6 Responsive Design**

The project specification requires a responsive interface so that the application can be used through different screen sizes.

### **5.7 Expected Outcome**

A usable web interface is created for entering information, viewing AI-generated plans, submitting feedback, and viewing stored records.

## PHASE 6 – DATABASE & FEATURE INTEGRATION

### 6.1 Introduction

This phase combines the frontend, backend, AI services, and database into an integrated FitBuddy application.

### 6.2 Database Integration

SQLite and SQLAlchemy are integrated with the FastAPI backend to store user information and generated workout plans. Database functions support saving, retrieving, and updating records.

### 6.3 AI and Database Flow

When a user submits fitness details, the backend sends the required information for AI generation, receives the generated plan, and stores relevant information. Later feedback can be processed to update the plan.

### 6.4 Feature Integration

Workout generation, nutrition/recovery guidance, feedback-based updates, and the user dashboard are connected through the defined FastAPI routes and backend functions.

### 6.5 End-to-End Workflow

The overall workflow is: user input → FastAPI request → Gemini processing → generated fitness plan → database storage/display → user feedback → updated plan.

### 6.6 Expected Outcome

All major FitBuddy features operate as connected components instead of isolated modules.

## PHASE 7 – TESTING & DEPLOYMENT

### 7.1 Introduction

This phase verifies the application functions and prepares it for local or deployment use.

### 7.2 Functional Testing

The main functions to verify include submitting user details, generating a workout plan, displaying results, submitting feedback, updating the plan, generating nutrition or recovery tips, storing records, and viewing users.

### 7.3 API Testing

FastAPI provides an interactive API documentation interface through /docs. The project can use this interface to inspect and test available backend endpoints.

### 7.4 Database Testing

Database operations should be checked to confirm that user and plan information can be saved, retrieved, and updated correctly.

### 7.5 User Interface Testing

The input form, result page, feedback function, dashboard, and responsive layout should be checked for correct display and interaction.

### 7.6 Deployment Preparation

The specification identifies environment configuration, Gemini API key setup, local server execution, and deployment considerations. Sensitive API configuration should be kept in environment variables rather than directly in application code.

### 7.7 Expected Outcome

The application is tested across its major workflows and prepared for running through the configured FastAPI server or deployment environment.

## PHASE 8 – FINAL EVALUATION, DOCUMENTATION & CONCLUSION

### 8.1 Introduction

The final phase evaluates the completed FitBuddy application against the project objectives and documents the overall implementation.

### 8.2 Final Feature Verification

The final system should demonstrate personalized workout generation, nutrition or recovery guidance, feedback-based plan updates, database storage, FastAPI backend services, Jinja2 frontend pages, and the user or coach dashboard.

### 8.3 Project Evaluation

The application is evaluated by checking whether the defined requirements and user scenarios are supported by the implemented modules. The complete workflow from fitness input to generated plan and feedback update is reviewed.

### 8.4 Documentation

The project documentation records the problem, proposed solution, requirements, architecture, AI integration, backend routes, frontend pages, database integration, testing, and deployment process.

### 8.5 Expected Outcome

The final outcome is a complete AI-powered web application concept that demonstrates the integration of FastAPI, Gemini AI, SQLite/SQLAlchemy, and Jinja2 for personalized fitness planning.

### 8.6 Conclusion

FitBuddy demonstrates how Generative AI can be integrated into a web application to create personalized fitness plans. The project combines AI-based workout generation with nutrition or recovery guidance, feedback-based updates, persistent data storage, and a dashboard. The phase-wise development process provides a structured path from ideation and requirements through architecture, implementation, testing, deployment, and final documentation.